

BUILD GUIDE — Shark Tank Market Map

How to rebuild this project from scratch (or adapt it to a different portfolio)

Purpose of this document: Drop it into a fresh Claude Code instance. Claude reads it and rebuilds the entire Shark Tank project from scratch, or adapts it to a different portfolio, broker, or data source. Every section is accurate to the actual repo as of June 2026.

SECTION 1 — INTAKE QUESTIONNAIRE

Read this section first and ask the user EVERY question before writing a single line of code.

Before building anything, you need the following information. Ask all questions in a single message. Do not proceed until you have answers (or explicit "use the default" confirmation) for each one.

1.1 — GitHub & Hosting

1. **GitHub username** — what is it? (The repo URL will be `github.com/<username>/<repo-name>`.)
2. **Repository name** — what should the repo be called? (Default: `shark-tank`)
3. **Public or private?** — GitHub Pages requires public for free tier; Vercel works with either.
4. **Hosting platform** — Vercel (recommended, zero-config), Netlify, GitHub Pages, or self-hosted? Default: Vercel.
5. **Custom domain?** — e.g. `harinidesai.com`. If yes, where is DNS managed (Namecheap, Cloudflare, GoDaddy)?
6. **Vercel account email** (if using Vercel) — so the deploy connection can be set up.

1.2 — Branding & Identity

1. **Site / brand name** — what appears in the nav bar? (Default: "Shark Tank · Market Map")
2. **Client display name** — what do you call the portfolio on the rendered pages? (Default: "Fable")
3. **Internal alias** — the alias used in the JSON file, different from display name for privacy. (Default: "Aurora")
4. **Brief timezone** — what timezone should the timestamp say? (Default: local system time.)

1.3 — Portfolio / Client Book

1. **Base currency** — EUR, USD, GBP, CHF? (Default: EUR)

2. **Holdings list** — for each position: ticker, asset class (equity/equity_etf/govt_bond/corp_bond/commodity_etc), quantity, entry date, entry price, currency of denomination, and any notes. These go into `client-book-json-[alias].json`.
3. **Liabilities** — any standing liabilities to net against the book (mortgages, loans)? Currency, monthly amount, end date.
4. **Client profile** — one sentence: who is this client? (MiFID Professional? Growth-oriented? Private bank? EMEA?) This text appears verbatim in the book file.
5. **TradingView ticker map** — for each holding, what is the TradingView symbol? (e.g. `"MU US": "NASDAQ:MU"`). Needed for live mark-to-market.

1.4 — Data Sources

1. **Live levels source** — TradingView scanner (default, no key needed) or manual? The scanner endpoint `https://scanner.tradingview.com/global/scan` works without authentication.
2. **RSI / OHLC data** — Yahoo Finance `/v8/finance/chart/` (default, no key needed). Any objection?
3. **Valuation multiples** — Yahoo Finance `/v10/finance/quoteSummary/` (default, no key). Any objection?
4. **Earnings data** — Finnhub (optional, requires a free API key). Do you have a Finnhub key? If yes, provide it; if not, the system falls back to web search for earnings estimates.
5. **Any additional paid data sources?** (Bloomberg, Refinitiv, IBKR Flex XML, etc.) If so, describe the format.

1.5 — Macro Book / Trades

1. **Initial trades.json** — start empty, or import existing positions? If importing: for each position provide the same schema as described in Section 4.5.
2. **Trade ID prefix** — default is `MM-YYYY-NNN` (e.g. `MM-2026-001`). Change?
3. **Conviction scoring weights** — use the standard 5-criterion rubric (gap/catalyst/positioning/confirmation/stop_quality = 3+2+2+2+1 = /10)? Or different weights?

1.6 — Pages & Tabs

1. **Which tabs do you want?** Default six: Summary, Insights, Earnings, Today's Watchlist, Portfolio, Derivatives Desk. Remove any?
2. **House-view authors** — the system cross-checks per-name views against named analysts. Default: JP Morgan GIS (institutional anchor) + Citrini Research, Doomberg, Michael Howell (CrossBorder Capital), Brent Donnelly (am/FX). Change any?

1.7 — Fact-Verification & Style

1. **Verification sources** — default: Reuters, WSJ, Bloomberg, Yahoo Finance, Seeking Alpha, Barron's (2 must agree). Change?
2. **Narrative style** — default is the Campbell/Doomberg/Pozsar/Papic/Perkins/Burry voice as defined in PROMPT.md. Any changes to the persona?

1.8 — Refresh Cadence

1. **How often will you refresh?** Daily pre-market (default), intraday, or on-demand?
2. **Will Claude run the gen script automatically (Claude Code in terminal), or will you run it manually?**

SECTION 2 — PROJECT OVERVIEW

What Is This?

The Shark Tank Market Map is a **Claude-generated, statically-hosted daily pre-market intelligence brief** for a private bank analyst. Claude Code is the only "backend." There is no server, no database, no cloud function, and no cron job. Every refresh is a Claude Code session.

The "Claude Is the Runner" Model

The entire pipeline — data fetch, mark-to-market, RSI computation, narrative authoring, HTML rendering, git commit, push — runs inside a single Claude Code session. Claude reads PROMPT.md (the standing brief specification), executes the instructions, updates the relevant Python script with today's fresh content, then runs it to regenerate all six HTML pages. A single `git push` triggers an instant Vercel deploy.

Human effort per refresh: ~5 minutes (open Claude Code, say "run the Market Map", review and approve the push).

High-Level Data Flow

```
PROMPT.md (brief spec) ■ ▼ Claude Code session ■ ■■ Web search (Reuters / Bloomberg / Yahoo Finance / etc.) ■ ■■ Macro news sweep (16+ searches) ■ ■■ Earnings data (Finnhub / web) ■ ■■ Geopolitical scan ■ ■■ gen_YYYY_MM_DD.py ← authored/updated by Claude each refresh ■ ■ ■ ■■ live_levels.py → TradingView scanner (live OHLCV) ■ ■■ fetch_rsi.py → Yahoo Finance (1y OHLC, RSI, TA, screener) ■ ■■ book_scanner.py → client-book-json-[alias].json + house_views.json ■ ■■ book.py → mark_to_market(), regime_log, trades.json I/O ■ ■■ shark_format.render_all() → index.html, insights.html, earnings.html, trades.html, portfolio.html, ideas.html, frag/*.html ■ ■■ git add / commit / push → Vercel auto-deploy (< 30 seconds)
```

Static Site, Zero Backend

All six pages are pre-rendered HTML files served directly by Vercel as static assets. The TradingView widget in the right rail fetches live quotes client-side via an official TradingView embed — no key needed. Everything else is baked into the HTML at render time.

SECTION 3 — TECH STACK & DEPENDENCIES

Python

- **Python 3.8+** — standard library only; no pip installs required.
- Libraries used from stdlib: `json`, `os`, `re`, `html`, `datetime`, `urllib.request`, `urllib.error`, `time`.
- **No third-party packages.** No `requests`, no `pandas`, no `numpy`. The RSI, Bollinger Bands, and Fibonacci are all hand-rolled in `fetch_rsi.py`.

Front-End

- **Vanilla HTML + CSS + minimal inline JavaScript** — no framework, no build step, no Node.
- All CSS is inlined in the `<style>` block of each page (emitted by `shark_format.py`).
- The hamburger nav and accordion `<details>` elements use native browser behaviour — zero JavaScript for those interactions.
- One small JS snippet powers the TradingView widget embed (official embed code, no key).

Data Endpoints (all free, no key unless noted)

Endpoint	Used for	Auth
<code>https://scanner.tradingview.com/global/assets</code>	Live cross-asset levels (FX, rates, equities, commodities)	None
<code>https://query1.finance.yahoo.com/v8/finance/chart/{ticker}?range=1y&interval=1d</code>	1-year OHLC for RSI + TA computation	None
<code>https://query1.finance.yahoo.com/v8/finance/summary/{ticker}?modules=defaultKeyStatistics,summary</code>	Valuation multiples (P/E, fwd P/E, EV/EBITDA, P/S)	None
<code>https://finnhub.io/api/v1/...</code>	Earnings estimates, recommendation splits	Free API key

Hosting

- **Vercel** — zero-config static host. `vercel.json` sets `outputDirectory: "."` and `buildCommand: null`. Any push to `main` deploys instantly.

SECTION 4 — REPOSITORY FILE-BY-FILE

4.1 — `gen_YYYY_MM_DD.py` (e.g. `gen_2026_06_11.py`)

The daily "brief file." Claude writes or rewrites this file each refresh. It contains:

1. **All narrative content** — `brief` dict with ~30 keys: `regime`, `regime_note`, `summary_narrative`, `wrap`, `takeaways`, `scenarios`, `insights_layers`, `correlation_regime`, `vol_skew`, `sector_rv`, `positioning`, `funding`, `tape_missing`, `consensus`, `one_chart`, `catalyst_calendar`, `what_changes_mind`, `client_ammo`, `ideas_note`, `event_radar_note`, `burry_tell`, `book_outlook`, `book_aim`, `book_pnl`, `dominant_theme`, `earnings_summary`, `earnings_why`, `staleness`, `earnings_ideas`.
2. **Live-data fetches** — calls to `live_levels.fetch()`, `fetch_rsi.fetch_all()`, `fetch_rsi.run_screener()`, `fetch_rsi.fetch_all_ideas()`.
3. **Per-trade enrichments** — a `TRADE_ENRICHMENTS` dict keyed by trade ID, with `instrument`, `fundamental_thesis`, `catalysts`, `risks`, `breakdown_why` (per-criterion explanations).
4. **Dashboard rows** — `dashboard`, `rates_levels`, `staleness` lists.
5. **Screener notes** — `SCREENER_NOTES` dict: ticker → desk commentary overlaying the mechanical RSI read.
6. **Mark-to-market overrides** — option marks and derived levels not available from TradingView (e.g. option spreads).
7. **Orchestration** — calls `book.mark_to_market()`, `book_scanner.build_scan()`, `shark_format.render_all()`, `book.save_json()`.

One new file per day. Old gen files are preserved in the repo as an auditable history.

Key `brief` dict keys and their types:

```
brief = { "regime": str, # gold-pill tag, e.g. "Hot PPI +6.5% YoY; ECB Hiked to 2.25%"
"regime_note": str, # 2-4 para prose explanation of today's macro regime
"summary_narrative": str, # HTML prose for "The overnight read" section (no bullets/lists)
"wrap": str, # HTML flowing prose (Campbell-style, 6+ paragraphs)
"takeaways": list[str], # 6-8 HTML prose paragraphs (no <li>)
"scenarios": list[dict], # [{kind, label, pct, headline, body}] - Bull/Base/Bear
"insights_layers": str, # HTML prose for the detailed analytical map
"correlation_regime": str, # HTML prose: 3 correlation-break observations
"vol_skew": str, # HTML prose: vol surface / VIX term structure read
"sector_rv": str, # HTML prose: sector leads/lags + RV idea
"positioning": str, # HTML prose: crowd positioning observation
"funding": str, # HTML prose: SOFR / plumbing / Pozsar mechanic
"tape_missing": str, # HTML prose: 3 under-priced risks (no numbered list)
"consensus": str, # HTML prose: consensus bid + the strongest OFFER
"one_chart": str, # HTML prose: the one level that matters today
"catalyst_calendar": list[dict], # [{day, date, event, consensus, view, asymmetry, dir}]
"what_changes_mind": str, # HTML: per-trade canaries (no <ul>/<li>)
"client_ammo": list[dict], # [{q, a}] - 4-6 client Q&A pairs
"ideas_note": str, # HTML: entry discipline for live ideas
"event_radar_note": str, # HTML: catalyst count + pending/done status
"burry_tell": str, # Plain text: the structural thing nobody is pricing
"dominant_theme": str, # Plain text: one-sentence regime descriptor
"book_outlook": { # "So what for the book" - macro→portfolio seam
"commentary": str,
"outperform": list[{"name": str, "why": str}],
"underperform": list[{"name": str, "why": str}],
"watch": list[{"label": str, "text": str}],
},
"book_aim": str, # Plain text: the stated aim for the book this session
"book_pnl": dict, # {note, open_avg_pct, realised_avg_pct} - book-level P&L
"staleness": list[dict], # [{datum, source, asof, stale}] - data freshness log
"earnings_summary": str, # Plain text: one-line per name post/pre-print summary
"earnings_why": str, # Plain text: why these names were included
"earnings_ideas": list[dict], # earnings conviction cards (schema in Section 9)
"screen": dict, # RSI screener output from fetch_rsi.run_screener() }
```

4.2 — `shark_format.py`

The renderer. ~4,500 lines. Consumes `brief`, `trades`, `regime_log`, and `scan` and emits six HTML files plus `frag/*.html` fragments.

Key functions:

Function	Purpose
<code>render_all(brief, trades, regime_log, scan=None)</code>	Entry point; writes all pages and frags
<code>_page_summary(brief, scan)</code>	<code>index.html</code> — Summary tab
<code>_page_insights(brief)</code>	<code>insights.html</code> — detailed analytical map
<code>_page_earnings(brief)</code>	<code>earnings.html</code> — earnings intelligence
<code>_page_trades(brief, trades)</code>	<code>trades.html</code> — Today's Watchlist + ideas
<code>_page_portfolio(brief, scan)</code>	<code>portfolio.html</code> — client book, pie charts
<code>_page_ideas(brief, scan)</code>	<code>ideas.html</code> — Derivatives Desk
<code>_explain_drop(label, sub, body, open=False)</code>	Collapsible <code><details></code> accordion wrapper
<code>_h(label, sub=None)</code>	Section heading (<code><h2 class="section-h"></code>)
<code>_book_outlook(brief, scan)</code>	"So what for the book" block (prose, no bullets)
<code>_takeaways(items)</code>	Takeaways as prose paragraphs (no <code>/</code>)
<code>_conviction_legend()</code>	Conviction rubric explainer table
<code>_rsi_screener(screen, notes)</code>	Oversold/overbought screener columns
<code>_universe_block(screen)</code>	Full ~140-name universe table
<code>_jpm_gis_anchor_block()</code>	JPM GIS methodology explainer
<code>_trade_card(t, enrichments, rsi_data)</code>	Full 8-section trade accordion
<code>_view_engine_panel(scan)</code>	Per-name house-view tiles
<code>_product_shelf()</code>	Derivatives product shelf explainer
<code>_positioning_method()</code>	RSI/Bollinger/CoT crowding methodology

CSS colour variables (defined in `CSS` constant at top of file):

```
--bg: #fff --surface: #f7f7f5 --ink: #1a1a1a --ink-soft: #6b6b6b --ink-mute: #9a9a9a
--gold: #b8960c --red: #c0392b --green: #1a7a45 --line: rgba(0,0,0,.1) --font:
-apple-system, "Helvetica Neue", sans-serif --serif: Georgia, "Times New Roman",
serif
```

4.3 — `book.py`

Trade book engine + legacy HTML builder. No third-party imports. Key functions:

Function	Purpose
<code>load_trades()</code>	Reads <code>trades.json</code> ; returns <code>{"open": [...], "closed": [...]}</code>
<code>save_json(path, data)</code>	Writes any JSON file atomically
<code>mark_to_market(trades, levels)</code>	Updates <code>current_pnl_pct</code> on every open trade; checks stops/targets; respects <code>min_hold_days</code>
<code>trade_direction(t)</code>	Returns <code>+1</code> (long) or <code>-1</code> (short) by parsing trade text
<code>update_regime_log(log, regime, note)</code>	Appends today's regime string to <code>regime_log.json</code>
<code>ingest_ideas(trades, ideas, type)</code>	Appends new ideas to <code>trades["open"]</code>
<code>build_html(brief, trades, regime_log)</code>	Legacy full-page HTML (for <code>output.html</code>) — still called per run
<code>e(s)</code>	HTML-escapes a string (re-exported for use across modules)
<code>step(msg) / log(msg)</code>	Terminal progress output with timestamps

Constants:

```
TRADES_PATH = "trades.json" REGIME_PATH = "regime_log.json" OUTPUT_PATH =
"output.html"
```

4.4 — `book_scanner.py`

Client-book module. Reads the portfolio JSON, marks positions to live prices, scores the portfolio, and fires derivative ideas. ~1,500 lines.

Key function: `build_scan(brief)` → returns `scan` dict:

```
scan = { "client": {"display_name": str, "base_currency": str, ...}, "metrics": {
"total_eur": float, # total NAV in base currency "usd_pct": float, # % of book in USD
```

```
"largest": {"ticker": str, "weight_pct": float}, # top concentration "fired": int, #
number of derivative ideas that triggered today }, "counts": {"fired": int, "watch":
int, "suppressed": int}, "positions": [...], # marked-to-live position list "ideas":
[...], # fired derivative ideas (product + rationale) "views": [...], # per-name
house views from house_views.json "alloc": {...}, # allocation breakdown by asset
class, sector, region }
```

Key constants in `book_scanner.py`:

```
BOOK_PATH = "client-book-json-Aurora.json" VIEWS_PATH = "house_views.json" IVOL_PATH
= "ivol_history.json" CLIENT_DISPLAY = "Fable" # ← change to your client display name
TV_MAP = { "MU US": "NASDAQ:MU", "SPY US": "AMEX:SPY", # etc. – one entry per holding
} PRODUCT_META = { 1: ("Collar + SBL + decumulator", "Concentration hedge /
de-risk"), 2: ("T-bill ladder + FX forward strip", "FX liability hedge"), # etc. –
product number → (structure name, objective) }
```

4.5 — `trades.json`

The live trade book. Schema:

```
{ "open": [ { "id": "MM-2026-001", "opened": "2026-05-31", "type": "reactive", //
"reactive" | "pre-position" "asset_class": "FX", // FX | Rates | Equity | Commodity |
Vol "trade": "Short EURAUD spot", "structure": "outright", // outright | spread |
option | ratio "entry": 1.645, "stop": 1.662, "target": 1.61, "conviction": 7, //
/10, recomputed each refresh "conviction_breakdown": { "gap": 3, "catalyst": 1,
"positioning": 1, "confirmation": 1, "stop_quality": 1 }, "horizon": "weeks", //
"days" | "weeks" | "months" "min_hold_days": 0, // min holding period before stop can
trigger "thesis": "...", "current": 1.6473, // filled by mark_to_market()
"current_pnl_pct": -0.14, // filled by mark_to_market() "history": [ {"date":
"2026-05-31", "level": 1.645, "pnl_pct": 0.0, "status": "open"} ] }, "closed": [ {
"id": "MM-2026-006", "exit": { "date": "2026-06-08", "level": 192.50, "pnl_pct":
-12.1, "result": "STOPPED" // "STOPPED" | "TARGET" | "DISCRETIONARY" }, "days_held":
8 } ] }
```

4.6 — `house_views.json`

Per-holding house views. Updated by Claude when the brief changes. Schema per view entry:

```
{ "ticker": "MU US", "view": "LIKE", // "LIKE" | "NEUTRAL" | "REDUCE" | "AVOID"
"confidence": "sourced", // "sourced" | "estimated" "conviction": 7, // /10
"conv_breakdown": [ {"k": "Evidence", "v": 3, "max": 3, "why": "..."}, {"k": "House
alignment", "v": 2, "max": 3, "why": "..."}, {"k": "Regime fit", "v": 1, "max": 2,
"why": "..."}, {"k": "Valuation & risk", "v": 1, "max": 2, "why": "..."} ],
"rationale": "One-sentence headline why.", "detail": "Longer paragraph. The full
analytical read.", "evidence": [...], "sources": [{"name": str,
"type": str, "note": str}], "brief_consistency": "How this view relates to today's
brief regime.", "formed": "2026-06-09", "stale_after": "2026-06-16" }
```

`_meta.macro_anchors.jp_gis` describes the JP Morgan GIS house anchor used to consistency-check every view.

4.7 — `ivol_history.json`

Implied volatility history. Used by `book_scanner.py` to provide an ATM-IV seed for each equity holding without a live options feed. Schema:

```
{ "MU US": [ {"date": "2026-06-09", "atm_iv_30d": 0.62, "source": "Yahoo options
chain / Black-Scholes inversion"} ] }
```

Black-Scholes inversion on Yahoo option marks is the source; `fetch_ivol.py` fetches and appends this.

4.8 — `fetch_rsi.py`

RSI + TA engine. See Section 6 for the full scoring description. Key functions:

Function	Purpose
<code>_fetch_ohlc(ticker, rng="1y")</code>	Fetches 1y daily OHLC from Yahoo <code>/v8/finance/chart/</code>
<code>_rsi_series(closes, period=14)</code>	14-period Wilder RSI series
<code>rsi_verdict(rsi)</code>	"OVERBOUGHT" (≥ 70) / "OVERSOLD" (≤ 30) / "NEUTRAL"
<code>technicals(ohlc, direction)</code>	50/100/200 SMA + Bollinger(20,2) + Fibonacci(126-session)
<code>analyse_one(trade_id)</code>	Full RSI + TA analysis for one trade
<code>fetch_all(trade_ids)</code>	Bulk analysis for all open trades
<code>fetch_valuation_stats(ticker)</code>	P/E, fwd P/E, EV/EBITDA, P/S from Yahoo quoteSummary
<code>analyse_idea(idea_key)</code>	RSI + TA + valuation for a Derivatives Desk idea
<code>fetch_all_ideas()</code>	Bulk analysis for all ideas
<code>run_screener()</code>	Scans ~140-name <code>SCREEN_UNIVERSE</code> ; returns oversold/overbought lists

TRADE_TICKERS maps every trade ID to `(yahoo_ticker, direction, note)`. When adding a new trade, add a row here.

SCREEN_UNIVERSE is the curated ~140-name cross-asset universe for the screener. Grouped by sector/region. Each entry: `(yahoo_ticker, display_name, sector, region)`.

4.9 — `live_levels.py`

TradingView scanner client. No auth needed. The `SYMBOLS` dict maps friendly names to TradingView ticker symbols:

```
SYMBOLS = { "brent": "ICEEUR:BRN1!", "wti": "NYMEX:CL1!", "gold": "TVC:GOLD",  
"eurusd": "FX:EURUSD", "usdjpy": "FX:USDJPY", "us10y": "TVC:US10Y", "spx": "SP:SPX",  
"dax": "XETR:DAX", # ... ~22 symbols total }
```

`fetch()` returns `{name: {"close": float, "chg_pct": float, "chg_abs": float}}`.

`trade_levels(snap)` maps the snapshot onto open trade IDs. Derived levels (Brent-WTI spread, 2s10s, DAX/Nasdaq ratio) are computed here.

4.10 — `client-book-json-[alias].json`

The portfolio file. Rename for your client (e.g. `client-book-json-Smith.json`). Update `BOOK_PATH` in `book_scanner.py`. Schema:

```
{ "client": { "client_id": "DEMO-0001", "alias": "Aurora", "base_currency": "EUR",
"account_opened": "2021-06-14", "profile": "...", "as_of": "2026-06-09",
"fx_reference": { "EURUSD": 1.1540, "source": "live, 09-Jun-2026" } }, "positions": [ {
"id": "EQ-001", "name": "Micron Technology", "ticker": "MU US", "asset_class":
"equity", "currency": "USD", "quantity": 16000, "entry_date": "2021-07-12",
"entry_price": 79.20, "cost_basis": 1267200, "mark_price": 935.89, "mark_quality":
"verified_live", "market_value": 14974240, "pnl_pct": 1081.6, "notes": "...",
"scanner_expected_triggers": ["concentration_above_15pct", "large_unrealised_gain"] }
] }
```

Valid `asset_class` values: `equity`, `equity_etf`, `govt_bond`, `corp_bond`, `commodity_etc`.

Valid `scanner_expected_triggers`: `concentration_above_15pct`, `large_unrealised_gain`, `catalyst_within_30d`, `ivol_elevated`, `beta_anchor`, `candidate_index_hedge_underlier`.

4.11 — `PROMPT.md`

The standing brief specification. Read by Claude at the start of every session. Contains:

- Fact-verification rules (see Section 7)
- The 24-hour global sweep mandate
- The voice rules (Campbell/Doomberg/Pozsar/Papic/Perkins/Burry moves)
- The conviction scoring rubric (see Section 6)
- The full output-structure specification
- Section-by-section instructions for every page
- The trade bias rule ("no inherited bias toward any instrument")

Never modify `PROMPT.md` to add a standing trade bias. The rules in that file prevent Claude from repeating yesterday's positions without fresh data.

4.12 — `vercel.json`

```
{ "buildCommand": null, "outputDirectory": ".", "framework": null }
```

This tells Vercel: serve the repo root as a static site, no build step.

4.13 — Generated HTML Files

File	Tab	Generated by
<code>index.html</code>	Summary	<code>shark_format._page_summary()</code>
<code>insights.html</code>	Insights	<code>shark_format._page_insights()</code>

<code>sources.py</code>	Source-quality helpers (verified / estimated / unverified labels)
<code>main.py</code>	Legacy entry point (mostly superseded by the dated gen scripts)
<code>earnings_data.md</code>	Pre-fetched Finnhub data; read by Claude at the start of each session

SECTION 5 — DATA PIPELINE

Step-by-Step Flow

Step 1 — Regime context (Claude reads `regime_log.json`)

Claude reads the last entry to understand yesterday's regime. Today's brief is a continuation, not a fresh start.

Step 2 — Live levels (`live_levels.fetch()`)

Calls the TradingView scanner endpoint with a batch of ~22 symbols. Returns a `snap` dict of `{name: {close, chg_pct, chg_abs}}`. Called once at the top of the gen script and cached for the entire run. Option marks (put/call spreads) are not available here; they are manually overridden in the gen script.

Step 3 — RSI + TA computation (`fetch_rsi.fetch_all()`)

For each open trade, fetches 1 year of daily OHLC from Yahoo Finance. Computes:

- 14-period Wilder RSI on the classic absolute 30/70 convention.
- 50/100/200-day SMAs + golden/death cross detection.
- 20-day Bollinger Bands (mean $\pm$ 2 SD) with %B and bandwidth.
- Fibonacci retracement off the trailing 126-session (6-month) swing.
- A TA score (0–2) and a plain-English read.
- A crowd verdict: is the tape extended against our direction?

Step 4 — Idea RSI (`fetch_rsi.fetch_all_ideas()`)

Same RSI + TA + valuation multiples for every Derivatives Desk idea. Valuation data (P/E, fwd P/E, EV/EBITDA, P/S) from Yahoo `quoteSummary`.

Step 5 — RSI screener (`fetch_rsi.run_screener()`)

Iterates the ~140-name `SCREEN_UNIVERSE`. For each name, fetches closes and computes 14-period RSI. Buckets into `oversold` ($RSI \leq 30$) and `overbought` ($RSI \geq 70$). Returns `{oversold: [...], overbought: [...], scanned: int, errors: int}`. The regime change is also detected here: if `regime` differs from the last `regime_log.json` entry, it is logged.

Step 6 — Mark to market (`book.mark_to_market()`)

For each open trade in `trades.json`, looks up `levels[trade_id]` (from `live_levels.trade_levels(snap)`). Computes `current_pnl_pct` from entry and direction. Checks stop and target; if hit and `min_hold_days` has elapsed, closes the trade. Appends a dated history

entry.

Step 7 — Client book scan (`book_scanner.build_scan()`)

Reads `client-book-json-[alias].json`, marks each position to live (via `TV_MAP` → TradingView scanner), computes NAV in base currency, allocation pies, largest concentration. Reads `house_views.json` for per-name views. Reads `ivol_history.json` for IV seeds. Runs the observable scoring ruleset against each position to produce fired derivative ideas.

Step 8 — Render (`shark_format.render_all()`)

Receives `brief`, `trades`, `regime_log`, `scan`. Emits all six HTML pages and `frag/*.html`. The TradingView widget is embedded as a `<script>` in the right rail of every page.

Step 9 — Save (`book.save_json()`)

Writes the updated `trades.json` and `regime_log.json` back to disk.

Step 10 — Git push (automated by Claude)

```
git add -A && git commit -m "Add 2026-06-11 brief: [article title]" && git push
```

Vercel detects the push and re-deploys within ~20 seconds.

SECTION 6 — THE CONVICTION SCORING SYSTEM

Core Rubric (applies to every macro trade idea, /10 total)

Criterion	Max	What it measures
Gap — price vs fundamentals	3	How wide is the mispricing? 3 = gross; 2 = clear; 1 = marginal

Catalyst — proximity	2	Is there a specific near-dated event? 2 = hard date; 1 = plausible; 0 = none
Positioning — extremity	2	Is the crowd leaning the wrong way? 2 = extreme; 1 = notable; 0 = neutral
Confirmation — cross-asset	2	Do other markets corroborate? 2 = strong; 1 = mixed; 0 = contradicts

Stop quality	1	Is the stop a clean technical level (1) or a vague zone (0)?
Core total	/10	

Live Computed Additions (added every refresh via `fetch_rsi`)

Criterion	Max	Source
Technical analysis (TA)	2	50/100/200 SMA + Bollinger + Fibonacci, direction-scored. 2 = trend agrees AND entry not chasing exhausted band; 1 = mixed; 0 = chart contradicts the trade
RSI positioning	1	Classic absolute. $RSI \geq 70$ long = crowded against us = 0 (−1 penalty); $RSI \leq 30$ short = same; otherwise 1

Combined totals shown on badges:

- Macro trade ideas / live book: /13 (core 10 + TA 2 + RSI 1)
- Derivatives Desk ideas: /11 (base 8 + TA 2 + RSI 1; earnings base is /8)

RSI: The Absolute Convention (CRITICAL)

The system uses classic Wilder RSI on a 14-period, 30/70 absolute threshold. It does **not** average RSI against a rolling mean or compute standard-deviation-from-mean — that approach was previously tried and discarded because it washed out the signal.

RSI $\geq 70 \rightarrow$ OVERBOUGHT (extended; bearish **signal for** a long) RSI $\leq 30 \rightarrow$ OVERSOLD (washed out; bullish **signal for** a long) $30 < RSI < 70 \rightarrow$ NEUTRAL

A position is "crowded against us" — and scores 0 on the RSI criterion — when:

- RSI ≥ 70 and we are **long** (chasing an extended tape)

- $RSI \leq 30$ and we are **short** (shorting into a washed-out tape)

Technical Analysis Scoring Detail

Moving averages (trend score): Price above all three SMAs (50/100/200) with a golden cross ($50 > 200$) = uptrend (score +1 for long). Price below all three with a death cross ($50 < 200$) = downtrend (score +1 for short). Mixed = 0.

Bollinger Bands (entry quality): Price at upper band = overbought/extended entry for a long (penalty). Price at lower band = oversold/support entry for a long (bonus). Bandwidth tells you if the market is in a squeeze (low vol → breakout risk). Bollinger %B < 0.2 for a long entry = supportive; %B > 0.8 = chasing.

Fibonacci retracement (levels): Off the trailing 126-session (6-month) swing high/low. Key levels: 23.6%, 38.2%, 50%, 61.8%, 78.6%. Entry at a Fib support = technically clean. Entry in open air between levels = less precise.

TA score (0–2) summary:

- 2 = trend confirms AND entry is at a support / not chasing an exhausted move
- 1 = trend confirms OR entry is clean, but not both
- 0 = chart contradicts the trade direction

Earnings Conviction Rubric (separate, /8 max)

Pillar	Max	Criterion
Asymmetry signal	2	Implied move materially misprices historical avg OR estimate dispersion wide
Sell-side consensus alignment	2	Majority buy / upside to PT / positive revisions
Catalyst clarity	2	Pre: unpriced catalyst; Post: disproportionate reaction to print quality
Positioning & sentiment	2	Short interest $> 10\%$ float OR extreme positioning

Conviction labels: **High** (≥ 6 , no 0 pillar, all sourced), **High — data gap** (≥ 6 , no 0, ≥ 1 estimated), **Medium** (4–5 OR any 0 OR consensus unverified), **Low** (< 4 or ≥ 2 zeros → excluded).

SECTION 7 — THE FACT-VERIFICATION RULE

This rule governs **every** claim in every section. It is non-negotiable and lives in [PROMPT.md](#).

FACT VERIFICATION: Before stating any market event, earnings result, price, or macro outcome as fact in the brief, cross-check it across multiple reputable sources — Reuters, Wall Street Journal, Bloomberg, Yahoo Finance, Seeking Alpha, and Barron's. A claim may only be stated as fact if at least TWO of these sources agree. If fewer than two confirm it, either omit it or clearly mark it as unverified/pending (red source dot). Never write a forward/near-dated event in past tense.

Concrete rules:

1. If an event is scheduled for TODAY or in the future: write it as PENDING/upcoming, never as done/completed — even if consensus probability is 99%.
2. If an earnings number, price level, or macro data point appears: verify against at least two sources. Flag unverified figures with a red dot.
3. If you cannot verify a claim: omit it or mark it "unverified." Never fabricate precision — a number without a source is not a number.
4. Temporal language is definitive: "hiked", "cut", "announced" = the event is **done**. If it is not done, use "expected", "pending", "consensus", "priced in", "anticipated."
5. Never quote more than 14 words from any single source.
6. Never use more than one direct quote per source.

The red dot convention: In the rendered HTML, any datum that is unverified gets a visual marker. The `staleness` list in the brief tracks every data point with its source, timestamp, and `stale: bool`.

SECTION 8 — THE WRAP / NARRATIVE STYLE

Non-Negotiable: Prose, Not Lists

The Summary page, Insights page, and the wrap section must read as continuous flowing narrative prose. **No `<ul>/<li>` bullet lists anywhere in the narrative sections.** No `<ol>` numbered lists. No bold "1. / 2. / 3. / 4." enumeration inside prose paragraphs. The renderer enforces this at the HTML level — `_takeaways()` outputs `<p class="takeaway-p">` elements; `_book_outlook()` outputs `<p class="bo-item">` elements.

The Voice DNA (from PROMPT.md)

The brief uses five specific analytical moves, borrowed from named analysts:

Move	Source	How to apply
------	--------	--------------

The Campbell move	Colin Campbell (Citi)	Decompose the headline number. "Equities up 9% YTD sounds bullish until you find 10 stocks account for all the gains." Always land on: so what, who's wrong, what's the trade.
The Doomberg pivot	Doomberg (Substack)	Name the second-order effect consensus is missing. One sentence. Use it to open the wrap.
The Pozsar mechanic	Zoltan Pozsár	Trace any move back to a balance sheet, a flow, or a funding constraint. Plumbing beats narrative. One per brief in the funding section.
The Papic constraint	Marko Papic	Every macro move has a political constraint. Name it. "The ECB can cut but the Bundesbank will frame it as reckless before the German election."
The Perkins regime-call	Michael Pettis / Perkins	Be willing to name the regime and commit to it: "Late-cycle stagflation." "Fiscal dominance transition." Name it at the top, defend it throughout.
The Burry tell	Michael Burry	One structural observation per brief that nobody is looking at and will matter in 6 months. No trade attached — just the observation and the implication. Goes in "What the tape is missing."

Style Rules (verbatim from PROMPT.md)

- Lead every section with a **CLAIM**, not a status update.
- Strip these words entirely: *could, may, potentially, appears to, seems to, suggests, somewhat, relatively, broadly, amid, against the backdrop of, in the wake of, market participants, investors are watching.*

- Declarative sentences. Short. Then occasionally one long synthesising sentence.
- Name names. Not "tech stocks fell" — "Nvidia, ASML, TSMC each fell 3%+."
- Never "in my view" or "I think."
- Never restate what happened without saying what to do about it.
- One analogy per output, maximum.
- Never end an observation without a directional take.
- Never say "watch X" without a specific level or threshold.

Building the Wrap from a 24-Hour Sweep

Before writing a word: run at least 16 searches covering all five regions (Asia, Japan, US, Europe, UK), all asset classes (equities, FX, rates, credit, commodities, vol, funding), geopolitics (military/political/sanctions/chokepoints), Trump/leader newsflow (Truth Social, press gaggle, tariff announcements), and single-name surprises (CEO comments, earnings, M&A). The wrap synthesises what was found into 5–7 connected paragraphs.

SECTION 9 — PAGE-BY-PAGE SPEC

Summary ([index.html](#))

Left column (LHS):

1. Regime tag (gold pill)
2. Title + meta line
3. **"So what for the book"** — collapsed `<details>` dropdown at the top; contains `_book_outlook()` block (commentary prose + outperform/underperform/watch as prose paragraphs, no bullets)
4. Fallback banner
5. **"The overnight read"** — `summary_narrative` as flowing HTML prose (no numbered themes)
6. **"Today's takeaways"** — 6–8 prose paragraphs via `_takeaways()` (no `<ul>/<li>`)
7. **Scenarios** — Bull/Base/Bear three tiles (green/gold/red accent border)
8. **"What the tape is missing"** — `tape_missing` as flowing prose (no 1/2/3/4 numbering)
9. **Catalyst calendar** — table of upcoming events with consensus/view/asymmetry/direction

Right column (RHS, sticky):

- TradingView live market quotes widget
- Dashboard tiles (2×grid, ~20 instruments, live from TradingView scanner)
- Rates table
- Theme line

Insights ([insights.html](#))

Detailed analytical map written as seamless prose — no "Layer 1/2/3" labels. Weaves regime, counter-intuitive hook, and the priced-vs-not-priced gap together. Sections embedded as `<iframe>`

fragments from `frag/`:

- Correlation regime (`frag/correlation.html`)
- Vol & skew (`frag/volskew.html`)
- Sector RV (`frag/sectorrv.html`)
- Positioning & flows (`frag/positioning.html`)
- Consensus bid/offer (`frag/consensus.html`)
- Client-call ammo (`frag/talking.html`)

Also contains: one-chart-that-matters, what-changes-mind, staleness check, citations.

Earnings (`earnings.html`)

- Conviction rubric definition at top
- Universe filter explainer (market cap \$10bn+, US/Korea, Tech/Financials/Industrials/Utilities, 5-day pre / 3-day post window)
- Per-name gold-border tile with: conviction badge, 4-pillar scores, key bullets, what-moves-it, client talking point
- Earnings summary + why-these-names

Today's Watchlist (`trades.html`)

LHS:

1. **"How conviction & positioning are scored"** — collapsed dropdown
2. **"RSI screener"** — collapsed dropdown: oversold names (to go higher) + overbought (to fade)
3. **"Our trading universe"** — collapsed dropdown: the full ~140-name table
4. **"How the JPM GIS view feeds our house anchor"** — collapsed dropdown (3-step methodology)
5. **Trade idea cards** — each card is an 8-section `<details>` accordion:
 - Section 1: Entry / Stop / Target / Conviction
 - Section 2: Instrument description
 - Section 3: Fundamental thesis
 - Section 4: Catalysts list
 - Section 5: Risks
 - Section 6: Conviction breakdown (per-criterion WHY)
 - Section 7: RSI & technical analysis (live, from `fetch_rsi`)
 - Section 8: What changes my mind (specific threshold)
6. **"Idea selection"** — which ideas go into the book and why, which were left out

RHS: TradingView widget + live book trades table with P&L.

Portfolio (`portfolio.html`)

- Allocation pie charts (asset class, equity sector, region) — SVG, rendered by `charts.py`

- Holdings table with live marks, P&L%, unrealised gain, IVol
- **House view engine** — per-name view tiles (LIKE/NEUTRAL/REDUCE/AVOID with conviction breakdown)
- Links to Derivatives Desk and Today's Watchlist for methodology

Derivatives Desk ([ideas.html](#))

- "The universe — where these ideas come from" — collapsed dropdown
- "The product shelf" — collapsed dropdown: equity (RevCon/FCN/Phoenix/Digital review/Autocall/BREN/protected participation + OTC: accumulators/decumulators/call-put spreads/risk reversals/prepaid variable forwards/variance/vol) + FX (DCS/forwards/seagulls/puts/accumulators) + Rates (range accruals)
- "How we judge if a trade is crowded" — collapsed dropdown: RSI + Bollinger + CoT/flow + pain trade methodology
- Per-idea tiles: product type, objective, RSI/TA live signals, conviction badge
- "Recovering losses" — exact route per underwater position + structured product path

The [_explain_drop\(\)](#) Dropdown Pattern

Used throughout for collapsed sections:

```
def _explain_drop(label, sub, body, open=False): sub_html = f'<span
class="sh-sub">{e(sub)}</span>' if sub else "" return ( f'<details
class="explain-drop"{" open" if open else ""}>' f'<summary><span
class="ed-h">{e(label)}{sub_html}</span>' f'<span class="ed-cue"></span></summary>'
f'<div class="ed-body">{body}</div>' f'</details>' )
```

The `sub` renders as `display:block` beneath the label. The `open=False` default means all methodology dropdowns start closed. The trade card sections start open (`open=True`) on the active session's card.

Icon Legend (on Watchlist and Portfolio tabs)

Icon	Meaning
✓ (green)	Working — position is in profit
X (red)	Stopped — position hit stop or was closed
→ (muted)	Flat — position is near entry, no meaningful P&L
■ (red)	Position stopped during min-hold override

SECTION 10 — STEP-BY-STEP SETUP FROM SCRATCH

Prerequisites

```
# Verify Python 3.8+ is installed python --version # or python3 --version on
Mac/Linux # Install GitHub CLI (if not present) # Mac: brew install gh # Windows:
winget install GitHub.cli # Linux: see cli.github.com/manual/installation # Install
Vercel CLI (optional – you can also connect via Vercel dashboard) npm install -g
vercel
```

1. Create the Local Project

```
mkdir shark-tank cd shark-tank git init git branch -M main
```

2. Create the Core Files

Create these files (Claude will write the content based on your intake answers):

```
shark-tank/ ■■■ PROMPT.md ← copy from this repo verbatim, then customise ■■■
vercel.json ← see Section 4.12 ■■■ book.py ■■■ shark_format.py ■■■ book_scanner.py
■■■ fetch_rsi.py ■■■ live_levels.py ■■■ fetch_earnings.py ■■■ fetch_ivol.py ■■■
earnings.py ■■■ charts.py ■■■ sources.py ■■■ trades.json ← start: {"open": [],
"closed": []} ■■■ regime_log.json ← start: [] ■■■ house_views.json ← start:
{"_meta": {...}, "views": []} ■■■ ivol_history.json ← start: {} ■■■
client-book-json-[alias].json ← your portfolio ■■■ gen_YYYY_MM_DD.py ← Claude
writes this first day
```

3. Create the GitHub Repository

Via CLI:

```
gh auth login gh repo create shark-tank --public --source=. --remote=origin --push
```

Manual alternative:

1. Go to github.com → New repository
2. Name it **shark-tank**
3. Set to Public (required for free GitHub Pages; Vercel works with either)
4. Do not initialise with README (you already have local files)
5. Back in terminal:

```
git remote add origin https://github.com/YOUR_USERNAME/shark-tank.git git add -A git
commit -m "Initial commit: Shark Tank Market Map" git push -u origin main
```

4. Connect Vercel

Via CLI:

```
cd shark-tank vercel # Follow prompts: # → Set up and deploy? Yes # → Which scope?
(your account) # → Link to existing project? No # → Project name: shark-tank # → In
which directory is your code located? ./ # → Auto-detected settings: override? No
```

Via Vercel Dashboard (alternative):

1. Go to vercel.com → New Project
2. Import from GitHub → select **shark-tank**

3. Framework: Other (no framework)
4. Build command: (leave blank)
5. Output directory: `.` (a single dot)
6. Click Deploy

Vercel will assign a URL like `shark-tank-[hash].vercel.app`.

5. Set a Custom Domain (optional)

In Vercel Dashboard:

1. Project → Settings → Domains
2. Add your domain (e.g. `harinidesai.com`)
3. Vercel shows the DNS records to add

In your DNS provider (e.g. Cloudflare):

- Add a CNAME record: `@` or `www` → `cname.vercel-dns.com`
- Or an A record: `@` → `76.76.21.21` (Vercel's IP)

DNS propagation: typically 5–30 minutes.

6. First Refresh

```
# In the shark-tank directory, with Claude Code running: python gen_2026_06_12.py
```

Or tell Claude Code in the chat: "Run the Market Map." Claude will:

1. Do the web search sweep (16+ searches)
2. Write/update the day's gen script with the brief content
3. Execute the gen script (fetches live data, renders HTML)
4. Run `git add -A && git commit -m "..."` && `git push`
5. Vercel auto-deploys within ~20 seconds

7. Every Subsequent Refresh

The workflow is the same every day:

1. Open Claude Code in the `shark-tank` directory
2. Say "Run the Market Map" (or "refresh")
3. Claude reads `PROMPT.md`, sweeps the last 24 hours, writes `gen_YYYY_MM_DD.py`, runs it, commits, and pushes
4. Your site is live with today's brief

To make corrections:

- For fact corrections: tell Claude "CRITICAL: [fact] was wrong, correct it to [correct fact], update everywhere and push"
 - For structural changes: describe the change; Claude edits `shark_format.py` and re-runs the gen script
 - For style changes: update `PROMPT.md` (Claude reads it fresh each session)
-

SECTION 11 — ADAPTATION NOTES

Changing the Portfolio

1. **Replace** `client-book-json-Aurora.json` with your holdings (follow the schema in Section 4.10).
2. **Update** `CLIENT_DISPLAY` and `BOOK_PATH` in `book_scanner.py`.
3. **Update** `TV_MAP` in `book_scanner.py` — one entry per holding (TradingView symbol).
4. **Update** `EQUITY_SECTOR` and `ASSET_BUCKET` dicts in `book_scanner.py` for correct allocation pies.
5. **Reset** `house_views.json` — remove the demo holdings' views, add your own. Or let Claude regenerate them on first run.
6. **Reset** `ivol_history.json` to `{}` then run `fetch_ivol.py` once to populate.
7. **Update** `LIABILITIES` in `book_scanner.py` if the client has standing liabilities to net.

Changing the Branding

- `CLIENT_DISPLAY` in `book_scanner.py`: what appears on the Portfolio and Summary pages
- `BRAND` constant in `shark_format.py` (near the top of `CSS`): site header text
- `NAV` list in `shark_format.py`: tab labels and subtitles
- The `<title>` format: search for `Shark Tank` in `shark_format.py` and replace

Changing Data Sources

Live levels: Edit `SYMBOLS` in `live_levels.py`. TradingView symbol format: `EXCHANGE:TICKER`. Find symbols at tradingview.com/symbols/. If TradingView changes their scanner endpoint, update `SCAN_URL`.

RSI/OHLC: Edit `TRADE_TICKERS` in `fetch_rsi.py` to point to different Yahoo tickers. The Yahoo Finance URL format is <https://query1.finance.yahoo.com/v8/finance/chart/TICKER?range=1y&interval=1d>. Common Yahoo ticker formats: `EURUSD=X` (FX), `GC=F` (gold futures), `^TNX` (10Y yield), `BZ=F` (Brent).

IBKR Flex XML (alternative live levels): If using Interactive Brokers Flex, write a replacement `live_levels.py` that parses the Flex XML report. The interface contract is: `fetch()` returns `{name: {"close": float}}`. Everything downstream works unchanged.

Stooq.com (alternative price source): `PROMPT.md` references stooq.com as the pull-from-live source for Claude's web-search step (during authoring). `live_levels.py` uses TradingView for the automated mark-to-market — these are independent.

Changing the Screener Universe

Edit `SCREEN_UNIVERSE` in `fetch_rsi.py`. Each entry: `(yahoo_ticker, display_name, sector, region)`. The screener runs on every name in this list each refresh. Aim for liquid, large/mid-cap

names — illiquid tickers return inconsistent Yahoo data.

Changing the Conviction Rubric

The rubric lives in two places:

1. **PROMPT.md** — governs Claude's authoring behaviour
2. `shark_format._conviction_legend()` — what is rendered on the Watchlist page

Both must be updated consistently if you change weights.

Changing the Derivatives Product Shelf

Edit `PRODUCT_META` in `book_scanner.py` — maps idea number to `(structure_name, objective)`. Edit `_product_shelf()` in `shark_format.py` to update the rendered description. The shelf is intentionally a guideline, not a hard limit — `PROMPT.md` instructs Claude to suggest off-shelf structures when they genuinely fit, tagged "please check feasibility with your derivs team."

Changing the Fact-Verification Sources

Edit the first paragraph of the `FACT VERIFICATION` section in `PROMPT.md`. If your organisation has preferred sources (e.g. FT, Refinitiv, ECB press releases), replace or add to the list. The "2 must agree" rule is the critical constraint — do not lower this threshold.

Deploying to Netlify Instead of Vercel

Replace `vercel.json` with `netlify.toml`:

```
[build] publish = "." command = ""
```

In the Netlify dashboard: New site → Import from GitHub → set publish directory to `.` → Deploy.

Deploying to GitHub Pages

In your repo settings → Pages → Source: Deploy from branch → `main` → `/` (root). Note: GitHub Pages adds a processing delay vs Vercel's instant deploy. The site must be public.

APPENDIX A — COMPLETE `vercel.json`

```
{ "buildCommand": null, "outputDirectory": ".", "framework": null }
```

APPENDIX B — TRADES.JSON STARTER

```
{"open": [], "closed": []}
```

APPENDIX C — REGIME_LOG.JSON STARTER

```
[]
```

APPENDIX D — HOUSE_VIEWS.JSON STARTER

```
{ "_meta": { "engine": "House View Engine", "formed_session": "YYYY-MM-DD", "note":  
  "Views formed by Claude from web evidence and brief regime.", "stale_after":  
  "YYYY-MM-DD", "conviction_rubric": "Conviction /10 = Evidence (/3) + House alignment  
  vs JP GIS (/3) + Regime fit (/2) + Valuation & risk (/2).", "macro_anchors": {  
    "jp_gis": "JP Morgan GIS current house view – insert summary here.", "authors": [] }  
  }, "views": [] }
```

APPENDIX E — IVOL_HISTORY.JSON STARTER

```
{}
```

Document accuracy: verified against the live repo as of 2026-06-12. Every file path, schema, function name, and data source has been read from the actual codebase — not invented.